

Nsec1 (Semi-confined, steady, 1-layer flow in consecutive sections)

T.N.Olsthoorn

11/06/2026

1 Intro

Nsec1 is een analytisch model voor het berekenen van 1 dimensionale stationaire stroming in een aantal gekoppelde secties van een semi-ge-spannen pakket. Zie het als een doorsnede door een aantal polders, elk met eigen kD , eigen c -waarde en eigen polderpeil, h . Aan de uiteinden wordt ofwel een gegeven stijghoogte ϕ , danwel een gegeven stroming Q_x opgelegd. Op de knooppunten tussen de secties geldt continuïteit van de stijghoogte en de stroming, met dien verstande daar op deze knooppunten ook een stroom water Q kan worden geïnfiltreerd of onttrokken.

Nsec1 is de 1-laags variant van **nsecn** in Python, een vergelijkbaar analytisch model voor meer lagen, dat ik rond 2000 ontwikkelde in Matlab, maar dat nog altijd niet naar Python is vertaald.

nsec1 is zowel in een python file **nsec1.py** als in een notebook **nsec1.ipynb** geïmplementeerd onder de **tools/analytic** en **tools/analytic/notebooks** directories.

1.1 Afleiding

De stroming in een sectie tussen $0 \leq x \leq L$ wordt analytisch beschreven door

$$\phi - h = Ae^{-x/\lambda} + Be^{-(L-x)/\lambda}$$

Hierin is

ϕ [m] de stijghoogte in het watervoerende pakket

h [m] het lokale polderpeil

L [m] de lengte van de sectie

x [m] de coördinaat langs de sectie met $0 \leq x \leq L$

λ [m] de spreidingslengte $\lambda = \sqrt{kDc}$

c [d] de verticale weerstand van de deklaag

kD [m²/d] het doorlaatvermogen van het watervoerend pakket

A [m] constante te bepalen uit de randvoorwaarden op $x = 0$ en $x = L$

B [m] constante te bepalen uit de randvoorwaarden op $x = 0$ en $x = L$

Q_x [m²/d] de horizontale stroming in het watervoerende pakket in positieve x -richting.

Deze stroming volgt met door differentiëren van de stijghoogte naar x en vermenigvuldiging met $-kD$:

$$Q_x = \frac{kD}{\lambda} \left(Ae^{-x/\lambda} - Be^{-(L-x)/\lambda} \right)$$

Door invullen van de randvoorwaarden $\phi(0)$ of $Q_x(0)$ en $\phi(L)$ of $Q_x(L)$ krijgen we twee concrete vergelijkingen met twee onbekenden, waaruit A en B kunnen worden opgelost en vervolgens ϕ en Q_x voor elke x binnen het traject kunnen worden berekend.

2 Meerdere gekoppelde secties

Bij meer secties gelden twee dezelfde vergelijkingen voor elk daarvan, elk met hun eigen constanten A_i en B_i en natuurlijk ook L_i , kD_i , c_i en λ_i . Deze secties worden met elkaar verknoopt op de contactpunten. Op elk daarvan geldt continuïteit van de stijghoogte en continuïteit van de stroming.

2.1 Continuïteit van de stijghoogte op de knooppunten

De index i verwijst naar het sectienummer maar ook naar het linker beginpunt van de sectie. Het rechter punt heeft dan index $i + 1$.

Voor sectie $i - 1$ geldt

$$\phi(x) - h_{i-1} = A_{i-1}e^{-\frac{x-X_{i-1}}{\lambda_{i-1}}} + B_{i-1}e^{-\frac{X_{i-1}-x}{\lambda_{i-1}}}, \quad X_{i-1} \leq x \leq X_i$$

en voor sectie i ernaast

$$\phi(x) - h_i = A_i e^{-\frac{x-X_i}{\lambda_i}} + B_i e^{-\frac{X_i-x}{\lambda_i}}, \quad X_i \leq x \leq X_{i+1}$$

Op het punt i , het punt waar deze secties koppelen geldt dat de stijghoogte $\phi(X_i)$ in beide secties hetzelfde is

$$A_{i-1}e^{-\frac{L_{i-1}}{\lambda_{i-1}}} + B_{i-1} + h_{i-1} = A_i + B_i e^{-\frac{L_i}{\lambda_i}} + h_i$$

Dus, de continuïteit van de stijghoogte op knooppunt X_i leidt tot onderstaande vergelijking

$$A_{i-1}e^{-\frac{L_{i-1}}{\lambda_{i-1}}} + B_{i-1} - A_i - B_i e^{-\frac{L_i}{\lambda_i}} = h_i - h_{i-1}$$

Op ditzelfde punt geldt ook continuïteit van de stroming

$$Q_x = \frac{kD_{i-1}}{\lambda_{i-1}} \left(A_{i-1}e^{-x/\lambda_{i-1}} - B_{i-1}e^{-(X_i-x)/\lambda_{i-1}} \right)$$

$$Q_x = \frac{kD_i}{\lambda_i} \left(A_i e^{-x/\lambda_i} - B_i e^{-(x-X_i)/\lambda_i} \right)$$

De waterbalans voor punt X_i luidt: Stroming van links + stroming van rechts plus injectie Q_i is nul:

$$A_{i-1} \frac{kD_{i-1}}{\lambda_{i-1}} e^{-L_{i-1}/\lambda_{i-1}} - B_{i-1} \frac{kD_{i-1}}{\lambda_{i-1}} - A_i \frac{kD_i}{\lambda_i} + B_i \frac{kD_i}{\lambda_i} e^{-L_i/\lambda_i} = -Q_i$$

Uitgedrukt in matrixvorm en met, korthedshalve, $\kappa = \frac{kD}{\lambda}$, volgt hier de kernel, voor vergelijkingen geldend voor knooppunt i

$$\begin{bmatrix} e^{-\frac{L_{i-1}}{\lambda_{i-1}}} & 1 & -1 & -e^{-\frac{L_i}{\lambda_i}} \\ \kappa_{i-1} e^{-\frac{L_{i-1}}{\lambda_{i-1}}} & -\kappa_{i-1} & -\kappa_i & \kappa_i e^{-\frac{L_i}{\lambda_i}} \end{bmatrix} \times \begin{bmatrix} A_{i-1} \\ B_{i-1} \\ A_i \\ B_i \end{bmatrix} = \begin{bmatrix} h_i - h_{i-1} \\ -Q_i \end{bmatrix}$$

Bij elk knooppunt zijn dus 4 coëfficiënten betrokken. Bij het linker punt van de meest linkse sectie met index $i = 0$ vallen de termen met $i - 1$ weg. Dat zijn de twee linker kolommen van deze kernel-matrix, de coëfficiënten A_{i-1} en B_{i-1} en het polderpeil h_{i-1} . De kernel wordt daar, met $i = 0$, met ϕ_0 op X_0

$$\begin{bmatrix} -1 & -e^{-\frac{L_0}{\lambda_0}} \\ -\kappa_0 & \kappa_0 e^{-\frac{L_0}{\lambda_0}} \end{bmatrix} \times \begin{bmatrix} A_0 \\ B_0 \end{bmatrix} = \begin{bmatrix} h_0 - \phi_0 \\ -Q_0 \end{bmatrix}$$

Dit is direct de specificatie voor de randvoorwaarde op $x = X_0$, waarvan de eerste regel geldt wanneer daar de stijghoogte $\phi_0 = h_0$ is gegeven en de tweede regel wanneer daar de injectie $Q = Q_0$ is gegeven.

Het meest rechts gelegen knooppunt heeft index N (de meest rechts gelegen sectie heeft index $N - 1$). Hier vallen alle sectie-paramaters weg met index N . Wat voor dit randpunt X_N overblijft van de kernel is, met $i = N$ en ϕ_N op X_N

$$\begin{bmatrix} e^{-\frac{L_{N-1}}{\lambda_{N-1}}} & 1 \\ \kappa_{N-1} e^{-\frac{L_{N-1}}{\lambda_{N-1}}} & -\kappa_{N-1} \end{bmatrix} \times \begin{bmatrix} A_{N-1} \\ B_{N-1} \end{bmatrix} = \begin{bmatrix} -h_{N-1} + \phi_N \\ -Q_N \end{bmatrix}$$

Dit is de specificatie van de randvoorwaarde van $x = X_N$ waarvan de eerste regel geldt wanneer daar de stijghoogte ϕ_N is opgegeven en de tweede regel wanneer daar in plaats van de stijghoogte de stroming (injectie positief) als randvoorwaarde geldt.

3 Systeem matrix

In de voorgaand sectie zijn alle vergelijkingen van dit systeem bekend per knooppunt. Deze moeten nu in een systeem worden samengevat bestaande uit een systeemmatrix met de bekende sectie eigenschappen voor elk knooppunt, een kolom met de te bepalen coëfficiënten en een kolom die voortvloeit uit de continuïteitseisen op elk knooppunt.

Er moeten twee constanten per sectie worden bepaald. Bij N secties leidt dit tot een systeem van $2N$ vergelijkingen met $2N$ onbekenden. We hebben echter steeds twee vergelijkingen per knooppunt en met $N + 1$ knooppunten krijgen we $2(N + 1)$ vergelijkingen. Het aantal kolommen is 4 per knooppunt, waarbij er steeds twee overlappen. Alleen voor de twee uiterste knooppunten krijgen we maar 2 kolommen. Dat geeft $2(N + 1) - 2 = 2N$ kolommen. Dus de systeem matrix heeft $2N + 2$ rijen en $2N$ kolommen. Echter vanwege de linker randvoorwaarde is van de eerste twee rijen er maar een geldig (stijghoogte of stroming op punt 0) en evenzo is vanwege de rechter randvoorwaarde van de laatste twee rijen er maar een geldig afhankelijk of daar de stijghoogte danwel de stroming als randvoorwaarde wordt opgelegd. Hiermee blijven er in een berekening exact $2N$ vergelijkingen met $2N$ onbekenden over.

Het zelfde geldt voor de R kolom in de matrix vergelijking (kolom rechts van het '=' teken) met de bekende eisen voor elke vergelijking. Ook dit zijn er $2(N + 1)$ waarvan maar één van de eerste twee waarden en één van de laatste twee waarden geldt afhankelijk van de daar gekozen randvoorwaarden, zodat er exact $2N$ waarden in de RHS over blijven.

De systeem matrix wordt opgebouwd door steeds de bovengenoemde kernel matrix te berekenen en die in de systeem matrix te plaatsen. Voor het linker knooppunt worden alleen de twee rechter kolommen geplaatst; voor het rechter uiterste knooppunt alleen de twee linker kolommen. Voor het oplossen van het systeem in de vorm van $A \times C = R$ met A de systeem matrix, C de vector onbekende constanten en R de „right hand side” met constanten, wordt eerste een boolean vector „active” gegenereerd met met lengte $2(N + 1)$ met alle waarden True, waarna een van de twee bovenste en een van de twee onderste waarden False worden gemaakt om de regels met de randvoorwaarden die niet gelden uit te sluiten zowel in de systeem matrix als in de R vector. Op deze wijze berekend:

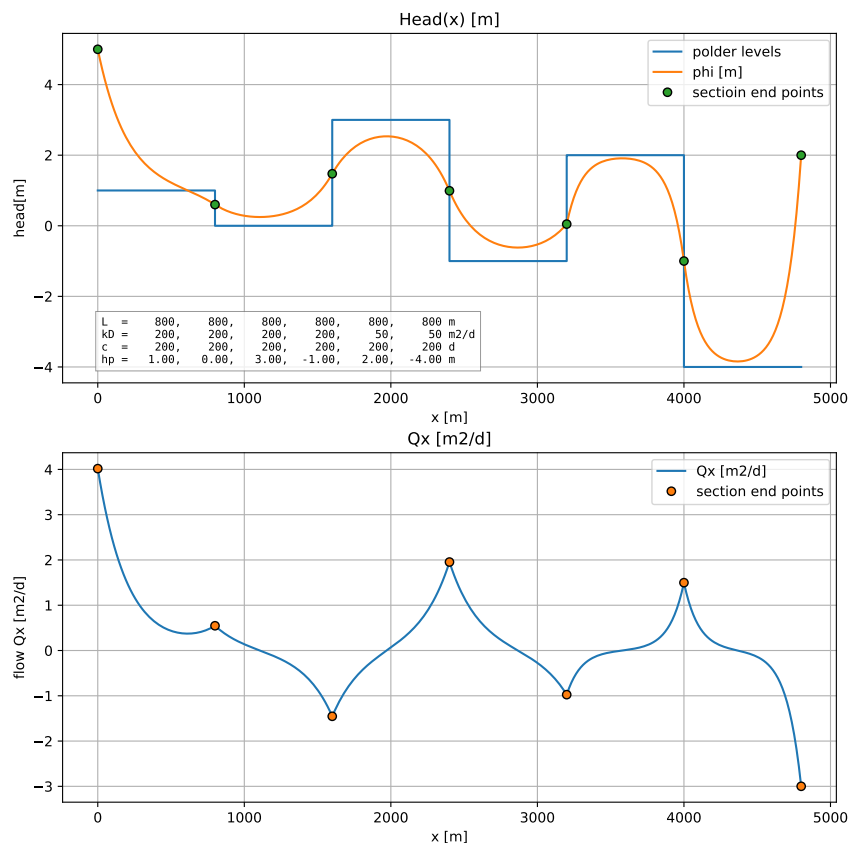
$$C = A[\text{active}]^{-1} R[\text{active}]$$

krijgen we exact de $2N$ coëfficiënten waarvan de even nummers, $C[:, 2]$, de A waarden zijn en de oneven nummers, $C[1 : 1]$, de B -waarden. Met deze nu bekende 2 coëfficiënten per sectie kunnen de stijghoogte en de stroming voor elke sectie voor elk punt binnen de sectie worden uitgerend onafhankelijk van alle andere secties. Door deze grafisch weer te geven blijkt dat zowel de stijghoogte als de stroming op de knooppunten continu zijn, dat de randvoorwaarden worden gehonoreerd en dat het verloop luistert naar de afzonderlijke polderpeilen en andere sectie-parameters.

4 Voorbeeld

Fig.1 geeft een voorbeeld van een berekening met nsec1. De uitgangsgegevens staan vermeld in de inzettable in de bovenste figuur. Deze toont het continue verloop van de berekende stijghoogte samen met de stijghoogte in de knooppunten en het „polderpeil. De onderste figuur geeft het berekende continue verloop

van de totale stroming in x -richting. Deze stroming is ook continue, maar de afgeleide niet. Waar de stroming 0 is, is de stijghoogtelijn horizontaal. Waar de stroming maximaal of minimaal is, is er een buigpunt in de stijghoogtelijn. Deze buigpunten liggen steeds op scheiding tussen twee polder waarvan minstens een van de sectie-eigenschappen verschilt met die ernaast.



nsec1.ipynb | 2026-06-11 18:23:09

Figuur 1: Met nsec1 berekend verloop van de stijghoogte en de stroming binnen een aantal koppelde secties, waarvan de gegevens in de inzet-tabel staan vermeld.

5 Listing

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from tools.etc import logo
5
6 NOTEBOOK_NAME = 'nsec1.ipynb'
7
8
9 """Analytic groundwater 1 layer steady 1D flow simulation in a series of adjaect polders.
```

```

11 Same as the old nsecn multilayer Matlab version, but now only 1 layer.

13 @TO 2026-06-11
14 """
15
16 class Nsec1():
17     """Class to analytically simulate stead state flow in a linked leaky aquifer.
18     The 1D single layer aquifer consits of sections of length L[i].
19     Each section has its now vertical resistance and its own transmissivity.
20     At each connection point a flow Q may be injected.
21     At both outer boundaries a head or a flow may be specified.
22     Afterwards the head and flow along all sections can can be simulated.
23
24     Simplified version of nsecn made in 2000.
25
26     @ TO 2026-0610
27     """
28
29     def __init__(self, kD=None, c=None, L=None, X0=0.0, Np=51):
30         self.kD = np.array(kD, dtype=float)
31         self.c = np.array(c, dtype=float)
32         self.L = np.array(L, dtype=float)
33         if not all(np.array([len(kD), len(c), len(L)]) == len(kD)):
34             raise ValueError("1D vectors kD, c and L must all have the same length.")
35         self.X0 = X0
36         self.Np = Np
37
38     @property
39     def X(self):
40         """Return section end points as array."""
41         return self.X0 + np.hstack((0, np.cumsum(self.L)))
42
43
44     @property
45     def x(self):
46         """Return x coordinates spanning the entire area.
47
48         You may inject the number of points per section by
49         injectiong the variable Np into the instantiated class.
50
51         mdl = Nsec1(...)
52         mdl.Np = 51
53
54         """
55         x = np.array([self.X[0]], dtype=float)
56         for x1, x2 in zip(self.X[:-1], self.X[1:]):
57             x = np.hstack((x, np.linspace(x1, x2, self.Np)[1:]))
58         return x
59
60     @property
61     def Nsec(self):
62         """Return number of sections."""
63         return len(self.kD)
64
65     @property
66     def lam(self):
67         """Return spreading lengths."""
68         return np.sqrt(self.kD * self.c)
69

```

```

71 # — Matrix coordinates kernel (submatrix)
72 def coefsAB(self, i):
73     """Return 2x4 kernel coefficient matrix truncated to 2x2 at both outer nodes.
74
75     Note 0 is the left node of section 0
76
77     """
78     N = len(self.kD)
79
80     eL = 0 if i < 0 else np.exp(-self.L[i-1] / self.lam[i-1])
81     eR = 0 if i >= N else np.exp(-self.L[i] / self.lam[i])
82
83     fL = 0 if i < 0 else self.kD[i-1] / self.lam[i-1]
84     fR = 0 if i >= N else self.kD[i] / self.lam[i]
85
86     M = np.array([
87         [eL, 1, -1, -eR],
88         [fL * eL, -fL, -fR, fR * eR],
89     ], dtype=float)
90     if i == 0:
91         M = M[:, 2:]
92     if i == N:
93         M = M[:, :2]
94     return M
95
96 @property
97 def coefMat(self):
98     """Return coefficient matrix."""
99     N = len(self.kD)
100     A = np.zeros((2*(N + 1), 2*N))
101     for i in range(N + 1):
102         M = self.coefsAB(i)
103
104         j0 = max(0, 2 * (i - 1))
105         j1 = min(2 * N, j0 + M.shape[1])
106
107         A[2*i:2*i+2, j0:j1] = M
108     return A
109
110 def set_boundaries(self, Phi=None, Q=None):
111     # — Boundary conditions
112     RHS = np.zeros(2*(self.Nsec + 1))
113     h = np.hstack((Phi[0], self.h, Phi[-1]))
114     RHS[::2] = + h[1:] - h[:-1]
115     RHS[1::2] = Q
116
117     # — Make Q equation or Phi equation at left and right inactive depending boundary type.
118     active = np.ones(2 * (self.Nsec + 1), dtype=bool)
119
120     if not np.isnan(Phi[0]):
121         # — Phi is given at left end
122         # — Q equation is inactive
123         # RHS[0] = -Phi[0]
124         active[0] = True
125         active[1] = False
126     else:
127         # — Q is given at left end
128         # — Phi equation is inactive
129         active[0] = False

```

```

129         active[1] = True

131     if not np.isnan(Phi[-1]):
132         # — Phi is given at right end
133         # — Q equation inactive
134         # RHS[-2] = Phi[-1]
135         active[-2] = True
136         active[-1] = False
137     else:
138         # — Q given at right end
139         # — Phi-equation inactive
140         active[-2] = False
141         active[-1] = True
142     return RHS, active
143
144 def simulate(self, Phi=None, Q=None, h=None, x=None, Np=51):
145     """Return heads and flow given the boundaries.
146
147     Parameters
148     -----
149     Phi: sequence of 2
150         Heads at left and right boundary. Use np.nan to use Q as boundary.
151     Q: None | sequence of 2 or sequence of Nsec + 1 (number of Nodes)
152         Flow boundary conditions. None to use zeros, seq. of 2 to use these
153         at either end and 0 for intermediate points. Use squence of Nsec + 1
154         to specify Q at all intermediate points.
155         Specificagion of Phi determines wheather the edge Q values will be used.
156     h: sequence of float (Nsec)
157         Adjacent head in each section ("polder level")
158     x: None or sequence of increasing x-coordinates
159         None will span x between self.X with Np points per section
160     Np: int
161         Number of coordinates per section if x is None
162         You may inject it into the instantiated class:
163
164         mdl = Nsec1(...)
165         mdl.Np = 51
166
167     """
168     self.Np = Np
169
170     if h is None:
171         self.h = np.zeros(self.Nsec)
172     else:
173         self.h = np.array(h, dtype=float)
174         if len(self.h) != self.Nsec:
175             raise ValueError(f"Number of heads must equal number of sections {self.Nsec}")
176
177     # — Boundary Q and injection at connection points
178     if Q is None:
179         Q = np.zeros(self.Nsec + 1)
180     else:
181         Q = np.array(Q, dtype=float)
182         assert len(Q) == 2 or len(Q) == self.Nsec + 1, f"Q must have length 2 or {self.Nsec + 1}"
183
184     # — Coordinates for plotting
185     if x is None:
186         x = self.x

```

```

189     # — Coefficient matrix
    A = self.coefMat

191
192     # — RHS
193     RHS, active = self.set_boundaries(Phi=Phi, Q=Q)

194
195     # — Solve for coefficients
    coefs = np.linalg.solve(A[active], RHS[active])

197
198     # — Compute heads and flows for all x
199     phi, Qx = self.get_phi_Q(coefs, x)

201
202     return {'phi': phi, 'Qx': Qx}

203 def get_phi_Q(self, coefs, x):
    """Return heads and flows."""
205     CA, CB = coefs[0::2], coefs[1::2]

206
207     # — Initialize heads and flows
    phi = np.zeros_like(x, dtype=float)
209     Q = np.zeros_like(x, dtype=float)

210
211     # — For each section compute heads and flows
    for i in range(self.Nsec):
213         mask = (x >= self.X[i]) & (x <= self.X[i+1])
        eL = np.exp(-(x[mask] - self.X[i]) / self.lam[i])
215         eR = np.exp(-(self.X[i+1] - x[mask]) / self.lam[i])
        phi[mask] = CA[i] * eL + CB[i] * eR + self.h[i]
217         Q[mask] = self.kD[i] / self.lam[i] * (CA[i] * eL - CB[i] * eR)
    return phi, Q

219
220
221 # — Aquifer traject properties
    L = np.array([800., 800., 800., 800., 800., 800.])
223     c = np.array([200., 200., 200., 200., 200., 200.])
    kD = np.array([800., 800., 800., 800., 200., 200.]) / 4
225     h = np.array([1., 0., 3., -1., 2., -4])

226
227 # — Q ————— Specify injections at every node including the outer two
    Q = np.array([0., 0., 0., 0., 0., 0.])

229
230 # — HEAD —————
231     Phi = np.array([5., 2.])

232
233
234     # — Coordinates of the nodes (must (or should be) increasing)
235     X = np.hstack((0, np.cumsum(L)))

236
237     # — Any set of coordinates will do. The nodes are included automatically.
    x = np.unique(np.hstack((np.linspace(0, np.sum(L), 161), np.cumsum(L))))

239
240
241     Xp = np.hstack((0, np.cumsum(L)))
    x = np.linspace(Xp[0], Xp[-1], 201)

243
244     mdl = Nsec1(kD=kD, c=c, L=L)
245     Xp = mdl.X

```



```

247 outx = mdl.simulate(Phi=Phi, Q=Q, h=h, x=None)
    outp = mdl.simulate(Phi=Phi, Q=Q, h=h, x=Xp)
249
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 10))
251 ax1.step(mdl.X, np.hstack((mdl.h, mdl.h[-1])), where='post', label='polder levels')
    ax1.plot(mdl.x, outx['phi'], label='phi [m]')
253 ax1.plot(Xp, outp['phi'], 'o', mec='k', label='section end points')

255 ax2.plot(mdl.x, outx['Qx'], label='Qx [m2/d]')
    ax2.plot(mdl.X, outp['Qx'], 'o', mec='k', label='section end points')
257
    # — Inset table with used properties
259 props_txt = [
        f"L = {'', '.join([f'{el:6.0f}' for el in L])} {'m':4}",
261        f"kD = {'', '.join([f'{kd:6.0f}' for kd in kD])} {'m2/d':4}",
        f"c = {'', '.join([f'{c_:6.0f}' for c_ in c])} {'d':4}",
263        f"hp = {'', '.join([f'{h_:6.2f}' for h_ in h])} {'m':4}",
    ]
265

267 ax1.text(
    0.05, 0.05, '\n'.join(props_txt),
269     transform=ax1.transAxes,
    ha='left', va='bottom',
271     family='monospace',
    fontsize=8,
273     bbox=dict(facecolor='white', edgecolor='0.5', linewidth=0.5, alpha=0.8)
)
275

277 ax1.set_title('Head(x) [m]')
    ax1.set_xlabel('x [m]', ylabel='head [m]')
279 ax1.grid(True)

281 ax2.set_title('Qx [m2/d]')
    ax2.set_xlabel('x [m]', ylabel='flow Qx [m2/d]')
283 ax2.grid(True)

285 ax1.legend()
    ax2.legend()
287
    logo(fig, NOTEBOOK_NAME)
289 images = '/Users/Theo/Development/python/hydro_tools/tools/analytic/images'
    fig.savefig(os.path.join(images, 'example_nsec1.pdf'))

```